Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Технології розподілених обчислень»
Комп'ютерний практикум №3
«Розробка паралельних
алгоритмів множення матриць та дослідження їх ефективності»

Виконав:
Студент III курсу
гр. ІП-33
Соколов О. В.

Прийняв:
Дифучин А. Ю.
"29" Березня 2026 р.

Київ – 2026

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №3

Тема: Розробка паралельних алгоритмів множення матриць та дослідження їх ефективності

Мета: Реалізувати стрічковий та алгоритм Фокса для множення квадратних матриць, провести експерименти з варіюванням розмірності матриць і кількості потоків, порівняти час виконання з послідовним алгоритмом та оцінити прискорення.

Завдання:

1. Реалізувати стрічковий алгоритм множення матриць. 30 балів.
2. Реалізувати алгоритм Фокса множення матриць. 30 балів.
3. Виконати експерименти з варіюванням розмірності матриць для обох алгоритмів, реєструючи час виконання; порівняти з послідовним алгоритмом; визначити прискорення в залежності від розміру матриць. 20 балів.
4. Виконати експерименти з варіюванням кількості потоків для паралельного множення, реєструючи час виконання; порівняти з послідовним алгоритмом; визначити прискорення в залежності від кількості потоків (та розміру матриць у складі експерименту). 20 балів.

Умови проведення замірів (з методичних вказівок): у час включати лише обчислення результату (без ініціалізації матриць), без виводу на консоль у нагрівальних і вимірювальних прогонах, 20 замірів на конфігурацію, розігрів одним прогоном послідовного алгоритму перед вимірюваннями, послідовний алгоритм вимірювати після паралельного для тієї ж конфігурації.

ВИКОНАННЯ

Завдання 1

Реалізовано стрічковий (рядковий) паралельний алгоритм у класі StripedParallelMultiplier (пакет parallel). Матриця результату C = A·B обчислюється так: діапазон рядків 0…n−1 розбивається на p смуг (кількість робочих потоків workers = min(p, n)), кожен потік обчислює усі елементи своїх рядків за стандартною формулою скалярного добутку рядка A на стовпці B. Потоки створюються як Thread, після start() для всіх виконується join() на кожному – синхронізація завершення без спільних структур даних під час обчислення (кожен потік пише у власні рядки C).

Коректність паралельної реалізації перевірено режимом --verify бенчмарку: для малих n результати збігаються з SequentialMultiplier у межах допустимої похибки для double.

Завдання 2

Реалізовано алгоритм Фокса у класі FoxParallelMultiplier (пакет parallel). Матриці розбиваються на блоки розміру m×m, де m = n/q і q – кількість блоків по рядку/стовпцю сітки; потрібно n % q == 0. На кожній з q фаз множення k = 0…q−1 для кожної пари блоків (bi, bj) запускається задача, яка додає добуток відповідних блоків A та B у блок C[bi][bj] (накопичення += у фазах). Паралелізм усередині фази – до q^2 задач у пулі ExecutorService з фіксованою кількістю потоків; між фазами використовується CountDownLatch, щоб усі блоки фази завершились перед початком наступної. Для бенчмарку кількість потоків узгоджується з параметром q: береться threads = q^2 (наприклад, 4, 9, 25 при q = 2, 3, 5).

Завдання 3

Експерименти з різною розмірністю n виконано за допомогою BenchmarkMain та BenchmarkRunner: матриці A, B заповнюються детерміновано (MatrixTestData.fillDeterministic) поза вимірюванням часу. Для кожної пари (алгоритм, n, кількість потоків) спочатку один розігрів послідовного множення, далі 20 вимірів паралельного часу, потім 20 вимірів послідовного часу; у звіт потрапляють середні значення t_parallel, t_sequential та прискорення S = t_sequential / t_parallel. Результати експортуються у CSV.

При зростанні n від приблизно 510 до 960 середній час послідовного алгоритму монотонно зростає (складність $O(n^3)$). Для фіксованої кількості потоків прискорення паралельних варіантів зазвичай також зростає разом із n: на малих матрицях накладні витрати на створення потоків і синхронізацію (особливо у схемі Фокса) відносно більші, на великих – корисна робота домінує і відношення часів стає вигіднішим.

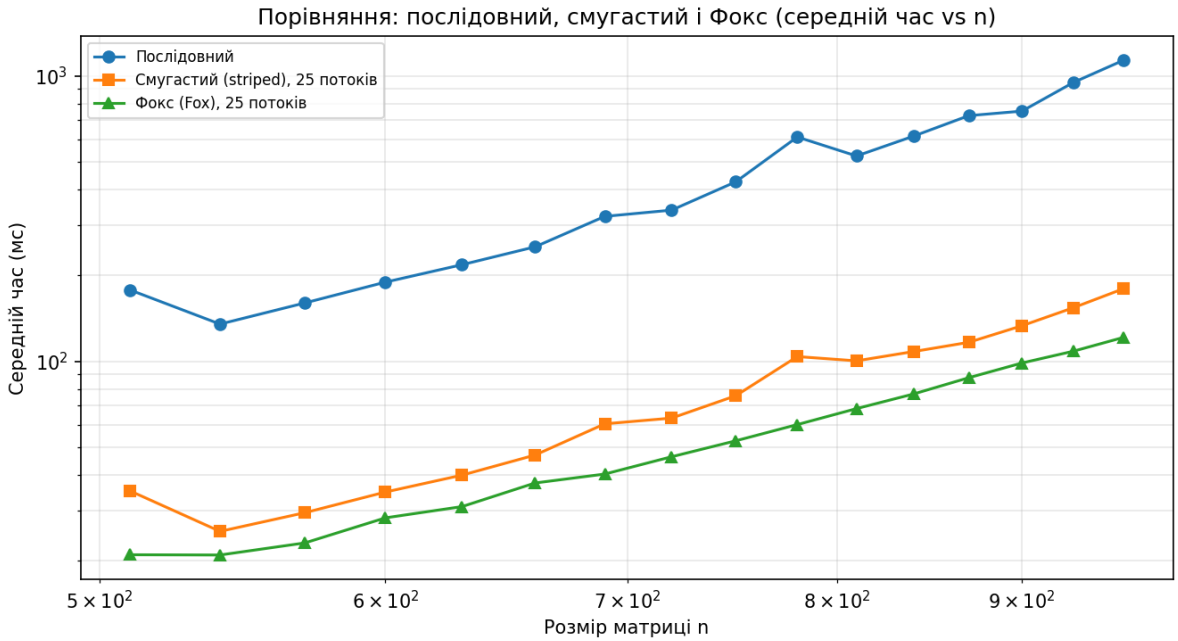

Рис. 3.1 – Залежність часу виконання від розміру матриці для послідовного, смугастого та алгоритму Фокса множення матриці

Завдання 4

Варіювалась кількість потоків: для стрічкового алгоритму – безпосередньо 4, 9, 25; для Фокса – відповідні q^2 з $q \in \{2,3,5\}$. Для кожного n порівнювались середні часи та прискорення відносно послідовного варіанту, виміряного після паралельного блоку для тієї ж конфігурації.

Спостереження:

- Стрічковий алгоритм: при переході з 4 на 9 і далі на 25 потоків прискорення зростає, але не лінійно відносно числа потоків – вплив витрат на керування потоками, балансування навантаження по рядках і обмежень заліза.
- Алгоритм Фокса: прискорення при більших q у багатьох випадках вище, ніж у стрічкового при тій же кількості потоків, але залежить від ділителя n на q і від фазових бар'єрів.

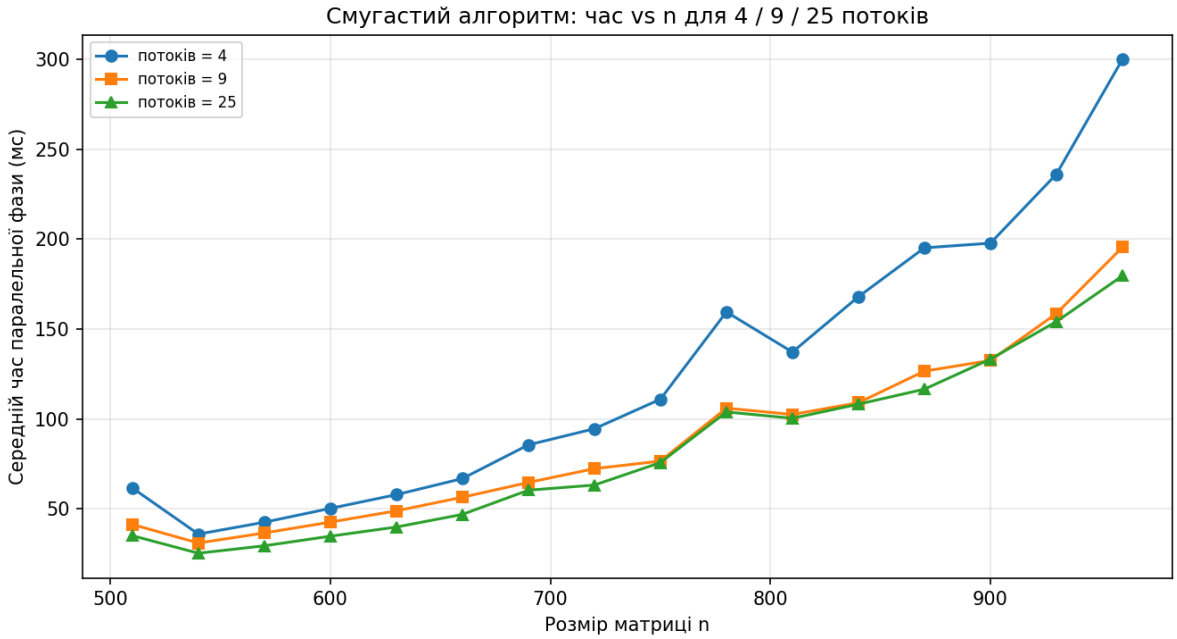

Рисунок 4.1 – Порівняння смугастого алгоритму із різною кількістю потоків

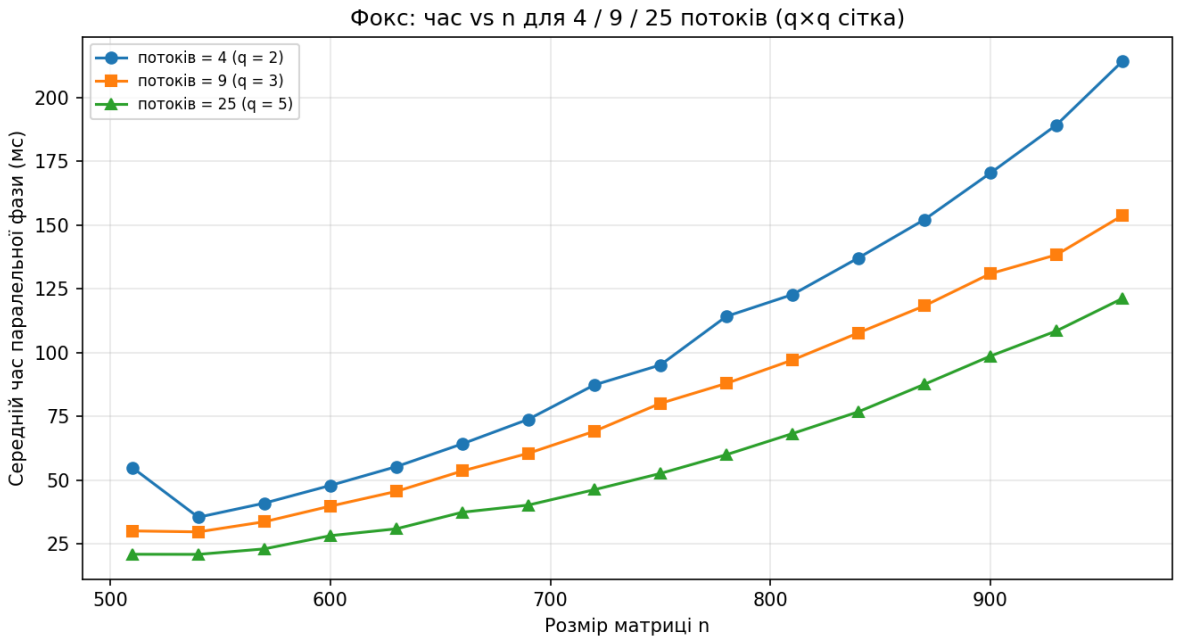

Рисунок 4.2 – Порівняння алгоритму Фокса із різною кількістю потоків

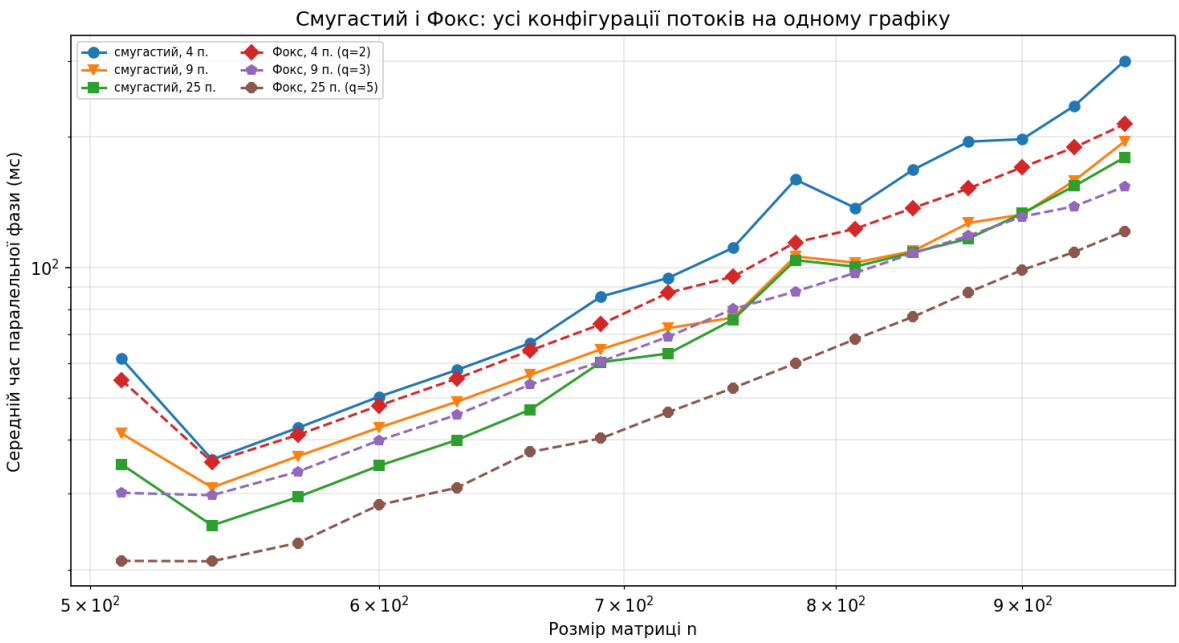

Рисунок 4.3 – Порівняння смугастого алгоритму та Фокса із різною кількістю потоків

Matrix Size	Sequential Algorithm	Striped Algorithm						Fox Algorithm					
		4 threads		9 threads		25 threads		4 threads		9 threads		25 threads	
	Time, s	Time, s	Speedup	Time, s	Speedup	Time, s	Speedup	Time, s	Speedup	Time, s	Speedup	Time, s	Speedup
510	0,177724292	0,061610525	2,858085	0,041343625	3,829741	0,035099944	4,563649	0,054962279	2,864374	0,030141346	5,221889	0,020985646	7,566805
540	0,135024615	0,035975708	3,855773	0,031013042	4,360289	0,025343354	5,342608	0,035499385	3,813616	0,029753869	4,571974	0,020944492	6,486699
570	0,160171717	0,042565619	3,7616	0,03660814	4,371936	0,029468581	5,417562	0,041008421	3,895846	0,033723556	4,814654	0,023089202	6,948487
600	0,189450304	0,05026991	3,769054	0,04263971	4,444757	0,034812594	5,436376	0,047946312	3,947574	0,039838156	4,755643	0,028266073	6,689833
630	0,218085427	0,057901742	3,768259	0,048899073	4,456478	0,039899675	5,469955	0,055317067	3,97891	0,045614377	4,803123	0,030953844	7,105988
660	0,251877279	0,066835162	3,774127	0,056459544	4,467123	0,046918652	5,382319	0,064257329	3,919573	0,053628367	4,84201	0,037449094	6,905129
690	0,322122448	0,085586065	3,787221	0,064667592	5,016624	0,06037595	5,396847	0,073866077	4,416062	0,060546931	4,862871	0,04025749	7,451458
720	0,338767792	0,094549225	4,00175	0,072362223	4,73455	0,063232096	5,346084	0,087339752	3,868782	0,069178269	5,059896	0,046288194	7,307864
750	0,425586329	0,11094906	3,81603	0,076558985	5,531438	0,075720662	5,590066	0,095149215	4,47537	0,080109331	4,906615	0,052636885	8,889897
780	0,611209692	0,159490858	3,86413	0,105979869	5,660207	0,103885846	5,853645	0,114195821	5,252046	0,08793016	5,608534	0,060005044	9,086198
810	0,52440809	0,137187269	3,857976	0,102476542	5,588846	0,100366617	5,701827	0,122731875	4,638366	0,09708621	5,109196	0,068263737	7,280723
840	0,615623038	0,167928117	3,667351	0,10900766	5,645411	0,108193467	5,712701	0,137107531	4,497557	0,107730302	5,826291	0,07682255	8,177214
870	0,726153827	0,195126706	3,774266	0,126586117	5,759139	0,116547311	5,693421	0,152118675	4,367299	0,118331242	5,501175	0,087584915	7,414639
900	0,753214354	0,197710256	3,819165	0,132490354	5,715199	0,133236871	5,663486	0,170360796	4,757377	0,130941892	6,200852	0,098639523	7,927825
930	0,949370583	0,236091923	3,82164	0,158603759	5,681412	0,154127269	5,846597	0,189192148	4,717765	0,138325694	6,475001	0,108500379	8,248244
960	1,134728273	0,2998968	3,802979	0,19545916	5,827377	0,17955056	5,983544	0,214162102	4,997524	0,153720127	6,976639	0,121216854	8,813432

Таблиця 4.1 – Результати тестування ефективності

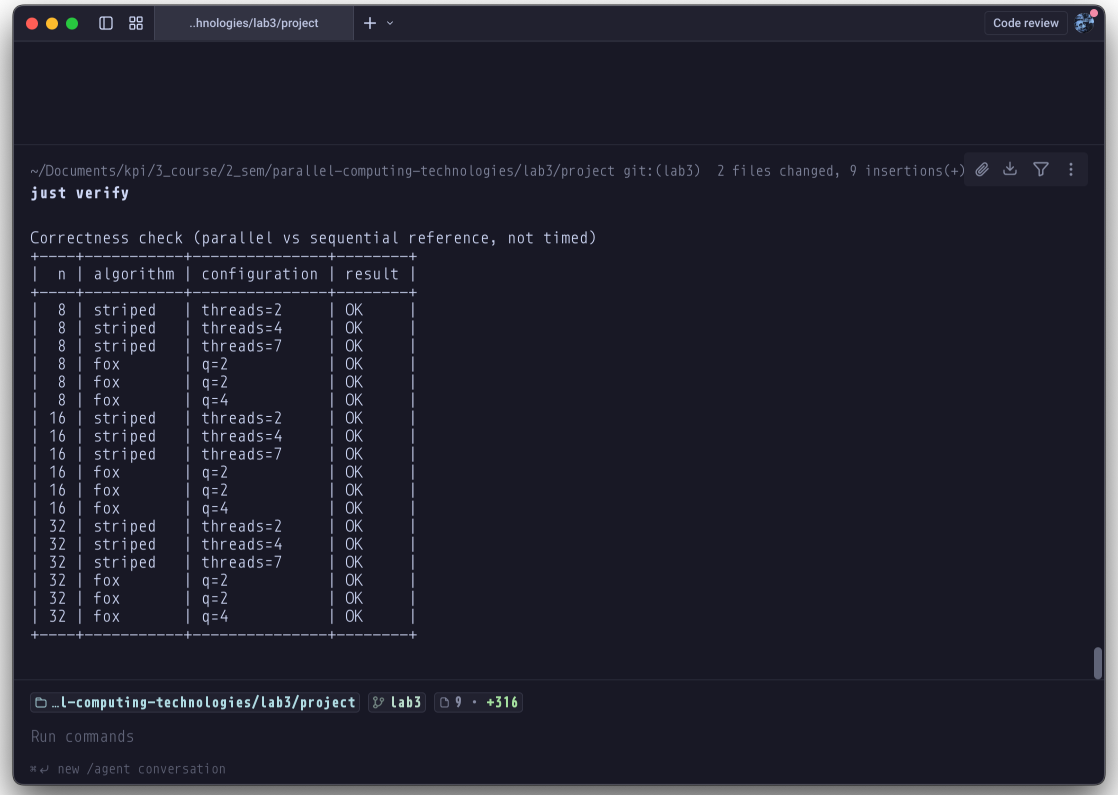

Рисунок 4.4 – Перевірка коректності алгоритмів

Висновок

У межах комп'ютерного практикуму №3 реалізовано два паралельні алгоритми множення квадратних матриць – стрічковий (StripedParallelMultiplier) та алгоритм Фокса (FoxParallelMultiplier), а також послідовний еталон (SequentialMultiplier). Налаштовано бенчмарк з протоколом, узгодженим із завданням: розігрів, по 20 замірів на фазу, паралельний блок перед послідовним, експорт у CSV.

Експерименти на діапазоні розмірів матриць і наборі потоків {4, 9, 25} показали стійке прискорення паралельних реалізацій відносно послідовної; прискорення зростає з n і з кількістю потоків, але не досягає ідеального лінійного масштабування через накладні витрати та архітектурні обмеження. На наведених даних алгоритм Фокса при q = 5 (25 потоків) часто давав найбільше прискорення серед порівнюваних конфігурацій.

Код програми

Посилання на GitHub репозиторій: https://github.com/sokolovgit/parallel-computing-technologies

```
===== ./benchmark/config/BenchmarkConfig.java =====
package config;

import java.nio.file.Path;
import java.util.Locale;

public final class BenchmarkConfig {

    private static final int DEFAULT_RUNS = 20;

    public final int[] sizes;
    public final int[] threads;
    public final int runs;
    public final Path outDir;
    public final String prefix;
    public final boolean sequential;
    public final boolean striped;
    public final boolean fox;
    public final boolean verify;

    public BenchmarkConfig(
            int[] sizes,
            int[] threads,
            int runs,
            Path outDir,
            String prefix,
            boolean sequential,
            boolean striped,
            boolean fox,
            boolean verify) {
        this.sizes = sizes;
        this.threads = threads;
        this.runs = runs;
        this.outDir = outDir;
        this.prefix = prefix;
        this.sequential = sequential;
        this.striped = striped;
        this.fox = fox;
        this.verify = verify;
    }

    public static BenchmarkConfig parse(String[] args) {
        int[] sizes = {512, 1024};
        int[] threads = {4, 9, 25};
        int runs = DEFAULT_RUNS;
        Path out = defaultOutDir();
        String prefix = "benchmark";
        boolean sequential = false;
        boolean striped = true;
        boolean fox = true;
        boolean verify = false;
```

```
        for (int i = 0; i < args.length; i++) {
            switch (args[i]) {
                case "--verify":
                    verify = true;
                    break;
                case "--sizes":
                    sizes = parseInts(require(args, ++i, args.length));
                    break;
                case "--threads":
                    threads = parseInts(require(args, ++i, args.length));
                    break;
                case "--runs":
                    runs = Integer.parseInt(require(args, ++i,
args.length));
                    break;
                case "--out":
                    out = Path.of(require(args, ++i, args.length));
                    break;
                case "--prefix":
                    prefix = require(args, ++i, args.length);
                    break;
                case "--algorithms":
                    String algs = require(args, ++i,
args.length).toLowerCase(Locale.ROOT);
                    if (algs.contains("all")) {
                        sequential = true;
                        striped = true;
                        fox = true;
                    } else {
                        sequential = algs.contains("sequential");
                        striped = algs.contains("striped") ||
algs.contains("both");
                        fox = algs.contains("fox") ||
algs.contains("both");
                    }
                    break;
                default:
                    throw new IllegalArgumentException("Unknown arg: " +
args[i]);
            }
        }
        if (!sequential && !striped && !fox) {
            throw new IllegalArgumentException(
                    "Choose at least one of: sequential, striped, fox (or
--algorithms all)");
        }
        return new BenchmarkConfig(sizes, threads, runs, out, prefix,
sequential, striped, fox, verify);
    }

    /**
     * Default is {@code ./results} from the JVM working directory. When
{@code -Dmatmul.home=DIR} is set
     * (the {@code just} recipes set it to the project root), output goes
to {@code DIR/results} so it stays
     * next to {@code matmul/} and {@code plots/}.
     */
    private static Path defaultOutDir() {
```

```
        String home = System.getProperty("matmul.home");
        if (home != null && !home.isEmpty()) {
            return Path.of(home).resolve("results");
        }
        return Path.of("results");
    }

    static String require(String[] args, int i, int len) {
        if (i >= len) {
            throw new IllegalArgumentException("Missing value");
        }
        return args[i];
    }

    static int[] parseInts(String s) {
        String[] parts = s.split(",");
        int[] out = new int[parts.length];
        for (int i = 0; i < parts.length; i++) {
            out[i] = Integer.parseInt(parts[i].trim());
        }
        return out;
    }
}

===== ./benchmark/runner/BenchmarkRunner.java =====
package runner;

import config.BenchmarkConfig;
import io.ResultsExporter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Locale;
import matrix.MatrixMultiplier;
import parallel.FoxParallelMultiplier;
import parallel.StripedParallelMultiplier;
import results.BenchmarkResult;
import sequential.SequentialMultiplier;
import util.MatrixOps;
import utils.MatrixTestData;

public final class BenchmarkRunner {

    private static final SequentialMultiplier SEQUENTIAL = new
SequentialMultiplier();

    private static final String[] BENCH_RESULT_HEADERS =
            new String[] {
                "algorithm",
                "n",
                "threads",
                "q",
                "t_parallel (ms)",
                "t_sequential (ms)",
                "speedup"
```

```
            };

    private static final boolean[] BENCH_RESULT_RIGHT =
            new boolean[] {false, true, true, true, true, true, true};

    private BenchmarkRunner() {}

    public static void runVerification() throws Exception {
        int[] sizes = {8, 16, 32};
        int[] stripedThreads = {2, 4, 7};
        int[] foxQ = {2, 2, 4};
        List<String[]> tableRows = new ArrayList<>();

        for (int n : sizes) {
            double[][] a = new double[n][n];
            double[][] b = new double[n][n];
            MatrixTestData.fillDeterministic(a, b);
            double[][] expected = new double[n][n];
            SEQUENTIAL.multiply(a, b, expected);

            for (int t : stripedThreads) {
                double[][] c = new double[n][n];
                MatrixOps.fillZero(c);
                new StripedParallelMultiplier(t).multiply(a, b, c);
                MatrixTestData.assertClose(expected, c, n, "striped n=" +
n + " t=" + t);
                tableRows.add(new String[] {String.valueOf(n), "striped",
"threads=" + t, "OK"});
            }

            for (int q : foxQ) {
                if (n % q != 0) {
                    continue;
                }
                double[][] c = new double[n][n];
                MatrixOps.fillZero(c);
                new FoxParallelMultiplier(q).multiply(a, b, c);
                MatrixTestData.assertClose(expected, c, n, "fox n=" + n +
" q=" + q);
                tableRows.add(new String[] {String.valueOf(n), "fox", "q="
+ q, "OK"});
            }
        }

        System.out.println();
        System.out.println("Correctness check (parallel vs sequential
reference, not timed)");
        printAsciiTable(
                new String[] {"n", "algorithm", "configuration",
"result"},
                tableRows,
                new boolean[] {true, false, false, false});
        System.out.println();
    }

    public static void runBenchmark(BenchmarkConfig cfg) throws Exception
{
        Files.createDirectories(cfg.outDir);
```

```
        String ts = Instant.now().toString().replace(':', '-');
        String base = cfg.prefix + "_" + ts;
        List<BenchmarkResult> rows = new ArrayList<>();

        String jvm = System.getProperty("java.version") + " " +
System.getProperty("java.vendor");
        String os = System.getProperty("os.name") + " " +
System.getProperty("os.version");
        String host = java.net.InetAddress.getLocalHost().getHostName();

        List<String[]> meta = new ArrayList<>();
        meta.add(new String[] {"Protocol", "mean " + cfg.runs + "
runs/phase; warmup 1× sequential per config"});
        meta.add(new String[] {"Output directory",
abbreviatePath(cfg.outDir.toAbsolutePath().normalize())});
        meta.add(new String[] {"Matrix sizes n",
formatIntArrayForMeta(cfg.sizes)});
        meta.add(new String[] {"Thread counts",
formatIntArrayForMeta(cfg.threads)});
        meta.add(new String[] {"Algorithms", formatAlgorithmsMeta(cfg)});
        meta.add(new String[] {"Host", host});
        meta.add(new String[] {"OS", os});
        meta.add(new String[] {"JVM", jvm});

        System.out.println();
        System.out.println("Matrix multiply benchmark");
        printAsciiTable(new String[] {"Setting", "Value"}, meta, new
boolean[] {false, false});

        List<String> skipNotes = new ArrayList<>();

        int[] benchWidths = benchResultFixedWidths();
        boolean streamingStarted = false;

        for (int n : cfg.sizes) {
            double[][] a = new double[n][n];
            double[][] b = new double[n][n];
            MatrixTestData.fillDeterministic(a, b);

            if (cfg.sequential) {
                BenchmarkResult seqRow = benchSequentialBaseline(a, b, n,
cfg.runs, jvm, os, host);
                rows.add(seqRow);
                streamingStarted =
                        printIntermediateBenchRow(benchWidths,
streamingStarted, seqRow);
            }
            if (cfg.striped) {
                for (int threads : cfg.threads) {
                    BenchmarkResult row = benchStriped(a, b, n, threads,
cfg.runs, jvm, os, host);
                    rows.add(row);
                    streamingStarted =
                            printIntermediateBenchRow(benchWidths,
streamingStarted, row);
                }
            }
            if (cfg.fox) {
```

```
                for (int threads : cfg.threads) {
                    int q = (int) Math.round(Math.sqrt(threads));
                    if (q * q != threads) {
                        skipNotes.add(
                                "n="
                                        + n
                                        + "  fox  threads="
                                        + threads
                                        + ": skip (need q² threads, e.g.
4, 9, 16)");
                        continue;
                    }
                    if (n % q != 0) {
                        skipNotes.add(
                                "n="
                                        + n
                                        + "  fox  threads="
                                        + threads
                                        + " (q="
                                        + q
                                        + "): skip (n not divisible by
q)");
                        continue;
                    }
                    BenchmarkResult row = benchFox(a, b, n, q, threads,
cfg.runs, jvm, os, host);
                    rows.add(row);
                    streamingStarted =
                            printIntermediateBenchRow(benchWidths,
streamingStarted, row);
                }
            }
        }

        if (streamingStarted) {
            System.out.println(horizontalRule(benchWidths));
            System.out.println();
        }

        List<String[]> resultLines = new ArrayList<>();
        for (BenchmarkResult r : rows) {
            resultLines.add(toResultLine(r));
        }

        if (resultLines.isEmpty()) {
            System.out.println();
            System.out.println("No benchmark rows (check algorithm flags
and thread / n constraints).");
        } else {
            System.out.println("Summary (all runs)");
            printAsciiTable(BENCH_RESULT_HEADERS, resultLines,
BENCH_RESULT_RIGHT);
        }

        if (!skipNotes.isEmpty()) {
            System.out.println();
            System.out.println("Skipped configurations");
            for (String note : skipNotes) {
```

```
                System.out.println("  • " + note);
            }
        }

        Path csvPath = cfg.outDir.resolve(base + ".csv");
        ResultsExporter.writeCsv(csvPath, rows);
        Files.copy(csvPath, cfg.outDir.resolve("latest.csv"),
java.nio.file.StandardCopyOption.REPLACE_EXISTING);
        System.out.println();
        System.out.println(
                "CSV: "
                        + csvPath.toAbsolutePath().normalize()
                        + "  →  "
                        +
cfg.outDir.resolve("latest.csv").toAbsolutePath().normalize());
    }

    /** Short summary so the settings table stays narrow (full list is
still in CSV / CLI). */
    private static String formatIntArrayForMeta(int[] values) {
        if (values.length == 0) {
            return "[]";
        }
        String full = Arrays.toString(values);
        if (full.length() <= 72) {
            return full;
        }
        if (values.length == 1) {
            return full;
        }
        int d = values[1] - values[0];
        boolean arithmetic = true;
        for (int i = 1; i < values.length; i++) {
            if (values[i] - values[i - 1] != d) {
                arithmetic = false;
                break;
            }
        }
        if (arithmetic) {
            return values.length
                    + " values: "
                    + values[0]
                    + " … "
                    + values[values.length - 1]
                    + " (step "
                    + d
                    + ")";
        }
        return values.length + " values: " + values[0] + " … " +
values[values.length - 1];
    }

    private static String formatAlgorithmsMeta(BenchmarkConfig cfg) {
        List<String> parts = new ArrayList<>();
        if (cfg.sequential) {
            parts.add("sequential");
        }
        if (cfg.striped) {
```

```
            parts.add("striped");
        }
        if (cfg.fox) {
            parts.add("fox");
        }
        return parts.isEmpty() ? "(none)" : String.join(" + ", parts);
    }

    private static String abbreviatePath(Path path) {
        String s = path.toString();
        final int max = 56;
        if (s.length() <= max) {
            return s;
        }
        int keep = (max - 3) / 2;
        return s.substring(0, keep) + " … " + s.substring(s.length() -
keep);
    }

    private static String[] toResultLine(BenchmarkResult r) {
        String qStr = r.q == null ? "-" : String.valueOf(r.q);
        return new String[] {
            r.algorithm,
            String.valueOf(r.n),
            String.valueOf(r.threads),
            qStr,
            String.format(Locale.US, "%.2f", r.tParallelMs),
            String.format(Locale.US, "%.2f", r.tSequentialMs),
            String.format(Locale.US, "%.2f", r.speedup),
        };
    }

    /** Column widths for streaming rows (fixed so layout stays stable
before later rows exist). */
    private static int[] benchResultFixedWidths() {
        int[] w = new int[BENCH_RESULT_HEADERS.length];
        for (int j = 0; j < w.length; j++) {
            w[j] = BENCH_RESULT_HEADERS[j].length();
        }
        w[0] = Math.max(w[0], 10);
        w[1] = Math.max(w[1], 6);
        w[2] = Math.max(w[2], 8);
        w[3] = Math.max(w[3], 4);
        w[4] = Math.max(w[4], 14);
        w[5] = Math.max(w[5], 14);
        w[6] = Math.max(w[6], 8);
        return w;
    }

    /**
     * Prints one intermediate result row; on first call prints section
title and table header.
     *
     * @return true once streaming has started (header printed)
     */
    private static boolean printIntermediateBenchRow(int[] widths, boolean
streamingStarted, BenchmarkResult row) {
        if (!streamingStarted) {
```

```
            System.out.println();
            System.out.println("Runs (intermediate, one row per
configuration as it finishes)");
            System.out.println(horizontalRule(widths));
            printTableRow(widths, BENCH_RESULT_HEADERS,
BENCH_RESULT_RIGHT);
            System.out.println(horizontalRule(widths));
        }
        printTableRow(widths, toResultLine(row), BENCH_RESULT_RIGHT);
        return true;
    }

    private static String horizontalRule(int[] widths) {
        StringBuilder rule = new StringBuilder("+");
        for (int width : widths) {
            rule.append("-".repeat(width + 2)).append('+');
        }
        return rule.toString();
    }

    /**
     * Plain ASCII table: column widths from header + cell content. {@code
rightAlign[j]} selects
     * right-aligned padding (for numbers).
     */
    private static void printAsciiTable(String[] headers, List<String[]>
rows, boolean[] rightAlign) {
        int cols = headers.length;
        int[] w = new int[cols];
        for (int j = 0; j < cols; j++) {
            w[j] = headers[j].length();
        }
        for (String[] row : rows) {
            for (int j = 0; j < cols; j++) {
                w[j] = Math.max(w[j], row[j].length());
            }
        }

        String line = horizontalRule(w);

        System.out.println(line);
        printTableRow(w, headers, rightAlign);
        System.out.println(line);
        for (String[] row : rows) {
            printTableRow(w, row, rightAlign);
        }
        System.out.println(line);
    }

    private static void printTableRow(int[] widths, String[] cells,
boolean[] rightAlign) {
        StringBuilder sb = new StringBuilder("|");
        for (int j = 0; j < cells.length; j++) {
            boolean right = rightAlign != null && j < rightAlign.length &&
rightAlign[j];
            String padded =
                    right
```

```
                            ? String.format(Locale.US, "%" + widths[j] +
"s", cells[j])
                            : String.format(Locale.US, "%-" + widths[j] +
"s", cells[j]);
            sb.append(' ').append(padded).append(" |");
        }
        System.out.println(sb);
    }

    /** Warmup ×1 sequential, then timed sequential-only runs (mean →
t_parallel = t_sequential, speedup 1). */
    private static BenchmarkResult benchSequentialBaseline(
            double[][] a, double[][] b, int n, int runs, String jvm,
String os, String host)
            throws Exception {
        double[][] cWarm = new double[n][n];
        SEQUENTIAL.multiply(a, b, cWarm);

        double[][] cSeq = new double[n][n];
        long[] sequentialNanos = new long[runs];
        for (int r = 0; r < runs; r++) {
            long t0 = System.nanoTime();
            SEQUENTIAL.multiply(a, b, cSeq);
            sequentialNanos[r] = System.nanoTime() - t0;
        }

        double meanSeq = meanNanos(sequentialNanos);
        double ms = meanSeq / 1_000_000.0;
        return new BenchmarkResult(
                "sequential",
                n,
                1,
                null,
                ms,
                ms,
                1.0,
                jvm,
                os,
                host);
    }

    private static BenchmarkResult benchStriped(
            double[][] a, double[][] b, int n, int threads, int runs,
String jvm, String os, String host)
            throws Exception {
        double[][] cWarm = new double[n][n];
        SEQUENTIAL.multiply(a, b, cWarm);

        MatrixMultiplier parallel = new
StripedParallelMultiplier(threads);

        double[][] cPar = new double[n][n];
        double[][] cSeq = new double[n][n];

        long[] parallelNanos = new long[runs];
        long[] sequentialNanos = new long[runs];

        for (int r = 0; r < runs; r++) {
```

```
        MatrixOps.fillZero(cPar);
        long t0 = System.nanoTime();
        parallel.multiply(a, b, cPar);
        parallelNanos[r] = System.nanoTime() - t0;
    }
    for (int r = 0; r < runs; r++) {
        long t0 = System.nanoTime();
        SEQUENTIAL.multiply(a, b, cSeq);
        sequentialNanos[r] = System.nanoTime() - t0;
    }

    double meanPar = meanNanos(parallelNanos);
    double meanSeq = meanNanos(sequentialNanos);
    double speedup = meanSeq / meanPar;

    return new BenchmarkResult(
            "striped",
            n,
            threads,
            null,
            meanPar / 1_000_000.0,
            meanSeq / 1_000_000.0,
            speedup,
            jvm,
            os,
            host);
}

private static BenchmarkResult benchFox(
        double[][] a,
        double[][] b,
        int n,
        int q,
        int threads,
        int runs,
        String jvm,
        String os,
        String host)
        throws Exception {
    double[][] cWarm = new double[n][n];
    SEQUENTIAL.multiply(a, b, cWarm);

    MatrixMultiplier parallel = new FoxParallelMultiplier(q);

    double[][] cPar = new double[n][n];
    double[][] cSeq = new double[n][n];

    long[] parallelNanos = new long[runs];
    long[] sequentialNanos = new long[runs];

    for (int r = 0; r < runs; r++) {
        MatrixOps.fillZero(cPar);
        long t0 = System.nanoTime();
        parallel.multiply(a, b, cPar);
        parallelNanos[r] = System.nanoTime() - t0;
    }
    for (int r = 0; r < runs; r++) {
        long t0 = System.nanoTime();
```

```
            SEQUENTIAL.multiply(a, b, cSeq);
            sequentialNanos[r] = System.nanoTime() - t0;
        }

        double meanPar = meanNanos(parallelNanos);
        double meanSeq = meanNanos(sequentialNanos);
        double speedup = meanSeq / meanPar;

        return new BenchmarkResult(
                "fox",
                n,
                threads,
                q,
                meanPar / 1_000_000.0,
                meanSeq / 1_000_000.0,
                speedup,
                jvm,
                os,
                host);
    }

    private static double meanNanos(long[] samples) {
        long sum = 0;
        for (long v : samples) {
            sum += v;
        }
        return (double) sum / (double) samples.length;
    }
}

===== ./benchmark/io/ResultsExporter.java =====
package io;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.util.List;
import java.util.Locale;
import results.BenchmarkResult;

public final class ResultsExporter {

    private ResultsExporter() {}

    public static void writeCsv(Path path, List<BenchmarkResult> rows)
throws IOException {
        StringBuilder sb = new StringBuilder();
        sb.append(

"algorithm,n,threads,q,t_parallel_ms,t_sequential_ms,speedup,jvm,os,hostna
me,timestamp\n");
        String ts = Instant.now().toString();
        for (BenchmarkResult r : rows) {
            sb.append(r.algorithm)
                    .append(',')
                    .append(r.n)
```

```
                    .append(',')
                    .append(r.threads)
                    .append(',')
                    .append(r.q == null ? "" : r.q)
                    .append(',')
                    .append(String.format(Locale.US, "%.6f",
r.tParallelMs))
                    .append(',')
                    .append(String.format(Locale.US, "%.6f",
r.tSequentialMs))
                    .append(',')
                    .append(String.format(Locale.US, "%.6f", r.speedup))
                    .append(',')
                    .append(escapeCsv(jvmShort(r.jvm)))
                    .append(',')
                    .append(escapeCsv(r.os))
                    .append(',')
                    .append(escapeCsv(r.hostname))
                    .append(',')
                    .append(ts)
                    .append('\n');
        }
        Files.writeString(path, sb.toString(), StandardCharsets.UTF_8);
    }

    private static String jvmShort(String jvm) {
        return jvm.replace('\n', ' ');
    }

    private static String escapeCsv(String s) {
        if (s.indexOf(',') < 0 && s.indexOf('"') < 0) {
            return s;
        }
        return '"' + s.replace("\"", "\"\"") + '"';
    }
}

===== ./benchmark/utils/MatrixTestData.java =====
package utils;

public final class MatrixTestData {

    private MatrixTestData() {}

    public static void fillDeterministic(double[][] a, double[][] b) {
        int n = a.length;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                a[i][j] = (i + 1) * 0.01 + (j + 1) * 0.001;
                b[i][j] = (i * n + j) * 0.0001 + 1.0;
            }
        }
    }

    public static void assertClose(double[][] expected, double[][] actual,
int n, String label) {
        double eps = 1e-8;
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
                double d = Math.abs(expected[i][j] - actual[i][j]);
                if (d > eps) {
                    throw new AssertionError(
                            label + " mismatch at (" + i + "," + j + "): "
+ expected[i][j] + " vs " + actual[i][j]);
                }
            }
        }
    }
}

===== ./benchmark/results/BenchmarkResult.java =====
package results;

/** One row of exported benchmark output. */
public final class BenchmarkResult {

    public final String algorithm;
    public final int n;
    public final int threads;
    public final Integer q;
    public final double tParallelMs;
    public final double tSequentialMs;
    public final double speedup;
    public final String jvm;
    public final String os;
    public final String hostname;

    public BenchmarkResult(
            String algorithm,
            int n,
            int threads,
            Integer q,
            double tParallelMs,
            double tSequentialMs,
            double speedup,
            String jvm,
            String os,
            String hostname) {
        this.algorithm = algorithm;
        this.n = n;
        this.threads = threads;
        this.q = q;
        this.tParallelMs = tParallelMs;
        this.tSequentialMs = tSequentialMs;
        this.speedup = speedup;
        this.jvm = jvm;
        this.os = os;
        this.hostname = hostname;
    }
}

===== ./benchmark/main/BenchmarkMain.java =====
package main;

import config.BenchmarkConfig;
import runner.BenchmarkRunner;
```

```
/**
 * CLI: benchmark striped and Fox multiplies; export CSV. Protocol per lab
task: warmup sequential
 * &times;1, then timed parallel runs, then timed sequential runs (per
configuration).
 */
public final class BenchmarkMain {

    private BenchmarkMain() {}

    public static void main(String[] args) throws Exception {
        if (args.length == 0) {
            printUsage();
            return;
        }
        BenchmarkConfig cfg = BenchmarkConfig.parse(args);
        if (cfg.verify) {
            BenchmarkRunner.runVerification();
            return;
        }
        BenchmarkRunner.runBenchmark(cfg);
    }

    private static void printUsage() {
        System.err.println(
                "Usage: BenchmarkMain --verify | [options]\n"
                        + "  --sizes n1,n2,...     matrix dimensions
(default 512,1024)\n"
                        + "  --threads t1,t2,...   thread counts for
striped (default 4,9,16)\n"
                        + "  --runs N              timed runs per phase
(default 20)\n"
                        + "  --out DIR             output directory
(default: ./results, or <matmul.home>/results if set)\n"
                        + "  --prefix NAME         file name prefix
(default benchmark)\n"
                        + "  --algorithms sequential|striped|fox|both|all
(default both; all = sequential+striped+fox)\n"
                        + "  --verify              correctness check
only\n");
    }
}

===== ./algorithms/parallel/StripedParallelMultiplier.java =====
package parallel;

import matrix.MatrixMultiplier;

/**
 * Row-band striped multiply (same approach as {@code
StripeMatrixMultiplication} in the course
 * reference): each thread owns a contiguous row range and computes full
dot products for those rows.
 */
public final class StripedParallelMultiplier implements MatrixMultiplier {

    private final int numProcessors;
```

```
    public StripedParallelMultiplier(int numProcessors) {
        this.numProcessors = numProcessors;
    }

    @Override
    public void multiply(double[][] a, double[][] b, double[][] c) throws
InterruptedException {
        int n = a.length;
        if (n == 0) {
            return;
        }

        int workers = Math.max(1, Math.min(numProcessors, n));
        Thread[] threads = new Thread[workers];
        int rowsPerThread = n / workers;
        int remainder = n % workers;
        int startRow = 0;
        for (int i = 0; i < workers; i++) {
            int rows = rowsPerThread + (i < remainder ? 1 : 0);
            int endRow = startRow + rows;
            final int sRow = startRow;
            final int eRow = endRow;
            threads[i] =
                    new Thread(
                            () -> {
                                for (int row = sRow; row < eRow; row++) {
                                    for (int j = 0; j < n; j++) {
                                        double sum = 0.0;
                                        for (int k = 0; k < n; k++) {
                                            sum += a[row][k] * b[k][j];
                                        }
                                        c[row][j] = sum;
                                    }
                                }
                            });
            startRow = endRow;
        }
        for (Thread thread : threads) {
            thread.start();
        }
        for (Thread thread : threads) {
            thread.join();
        }
    }
}

===== ./algorithms/parallel/FoxParallelMultiplier.java =====
package parallel;

import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import matrix.MatrixMultiplier;
import util.ExecutorUtils;

/**
```

```
 * Fox block algorithm on a {@code q&times;q} grid; {@code q}<sup>2</sup>
tasks per phase. Requires
 * {@code n % q == 0}.
 */
public final class FoxParallelMultiplier implements MatrixMultiplier {

    private final int q;

    public FoxParallelMultiplier(int q) {
        this.q = q;
    }

    @Override
    public void multiply(double[][] a, double[][] b, double[][] c) throws
InterruptedException {
        int n = a.length;

        if (n == 0) {
            return;
        }

        if (q < 1 || n % q != 0) {
            throw new IllegalArgumentException("n must be divisible by
q");
        }

        int m = n / q;
        int workers = q * q;

        ExecutorService executor = Executors.newFixedThreadPool(workers);
        try {
            for (int k = 0; k < q; k++) {
                CountDownLatch latch = new CountDownLatch(workers);
                final int kf = k;
                for (int bi = 0; bi < q; bi++) {
                    for (int bj = 0; bj < q; bj++) {
                        final int biF = bi;
                        final int bjF = bj;
                        executor.execute(() -> {
                            try {
                                int aRow0 = biF * m;
                                int aCol0 = ((biF + kf) % q) * m;
                                int bRow0 = ((biF + kf) % q) * m;
                                int bCol0 = bjF * m;
                                int cRow0 = biF * m;
                                int cCol0 = bjF * m;
                                blockMultiplyAdd(a, aRow0, aCol0, b,
bRow0, bCol0, c, cRow0, cCol0, m);
                            } finally {
                                latch.countDown();
                            }
                        });
                    }
                }
                latch.await();
            }
        } finally {
            ExecutorUtils.shutdownQuietly(executor);
```

```
        }
    }

    private static void blockMultiplyAdd(
            double[][] a,
            int aRow0,
            int aCol0,
            double[][] b,
            int bRow0,
            int bCol0,
            double[][] c,
            int cRow0,
            int cCol0,
            int m) {
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < m; j++) {
                double sum = 0.0;
                for (int kk = 0; kk < m; kk++) {
                    sum += a[aRow0 + i][aCol0 + kk] * b[bRow0 + kk][bCol0
+ j];
                }
                c[cRow0 + i][cCol0 + j] += sum;
            }
        }
    }
}

===== ./algorithms/util/ExecutorUtils.java =====
package util;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.TimeUnit;

public final class ExecutorUtils {

    private ExecutorUtils() {}

    public static void shutdownQuietly(ExecutorService executor) {
        executor.shutdown();
        try {
            if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
                executor.shutdownNow();
            }
        } catch (InterruptedException e) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}

===== ./algorithms/util/MatrixOps.java =====
package util;

/** Helpers for preparing result buffers. */
public final class MatrixOps {

    private MatrixOps() {}
```

```
    public static void fillZero(double[][] c) {
        for (int i = 0; i < c.length; i++) {
            for (int j = 0; j < c[i].length; j++) {
                c[i][j] = 0.0;
            }
        }
    }
}

===== ./algorithms/sequential/SequentialMultiplier.java =====
package sequential;

import matrix.MatrixMultiplier;

/** Sequential multiply (same structure as {@code
DefaultMatrixMultiplication} in course reference). */
public final class SequentialMultiplier implements MatrixMultiplier {

    @Override
    public void multiply(double[][] a, double[][] b, double[][] c) {
        int n = a.length;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                double sum = 0.0;
                for (int k = 0; k < n; k++) {
                    sum += a[i][k] * b[k][j];
                }
                c[i][j] = sum;
            }
        }
    }
}

===== ./algorithms/matrix/MatrixMultiplier.java =====
package matrix;

/**
 * Square matrix multiply {@code C = A &times; B} into {@code c}. Parallel
implementations may use
 * multiple threads; {@link InterruptedException} is declared for
compatibility with those paths.
 */
@FunctionalInterface
public interface MatrixMultiplier {

    void multiply(double[][] a, double[][] b, double[][] c) throws
InterruptedException;
}
```